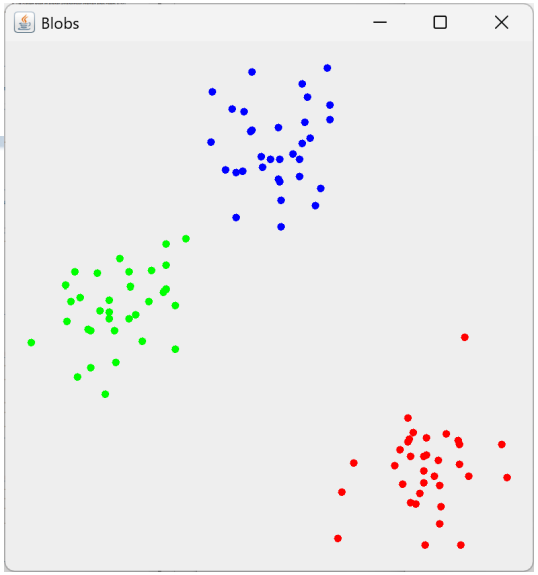
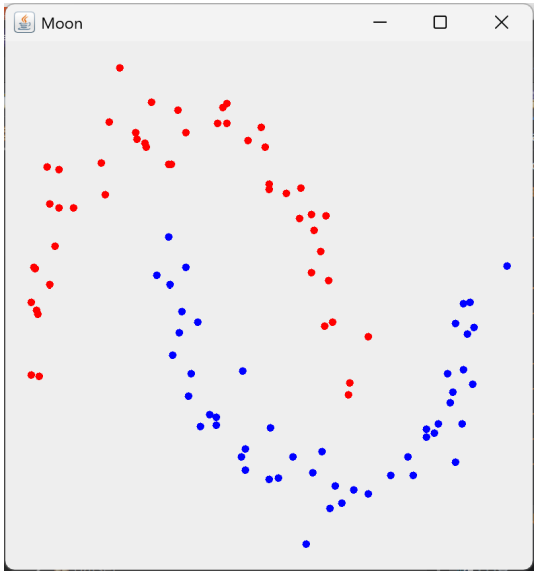
Aufgaben zur Abgabe als Prüfungsleistung für Prog II

- Regeln vergleichbar Prog I
  - Sprache: Alles Englisch
  - Die Klassen sollen im eigenen Abgabeprojekt nach eigenem Ermessen in Pakete eingeordnet werden
  - Verwenden Sie keine Umlaute wie ä, ö, ü oder Sonderzeichen wie ß
  - Zielumgebung: mind. Java 24 mit Eclipse mind. 2025-12
  - Plagiate (d.h. Kopien oder veränderte Kopien anderer Lösungen) werden mit 0 Punkten sowohl für das „Original“ als auch die „Kopie“ bewertet
- Eclipse Import: Im Package Explorer, rechte Maustaste: Import: General: Existing Projects into Workspace, Select Archive File (Browse ….zip), Finish
  Hinweis: Dieser Import setzt voraus, dass das Projekt noch nicht existiert, deshalb ggfs. ein altes Projekt in Eclipse vorher umbenennen
- Eclipse Export: Eclipse Projekt: Rechte Maustaste: Export: General: Archive File (dann: Save in zip Format, Finish)
- Punkteverteilung
  - 12 Punkte Funktionalität (inkl. Strategiemuster)
  - 8 Punkte Qualität

K-Means-Clustering

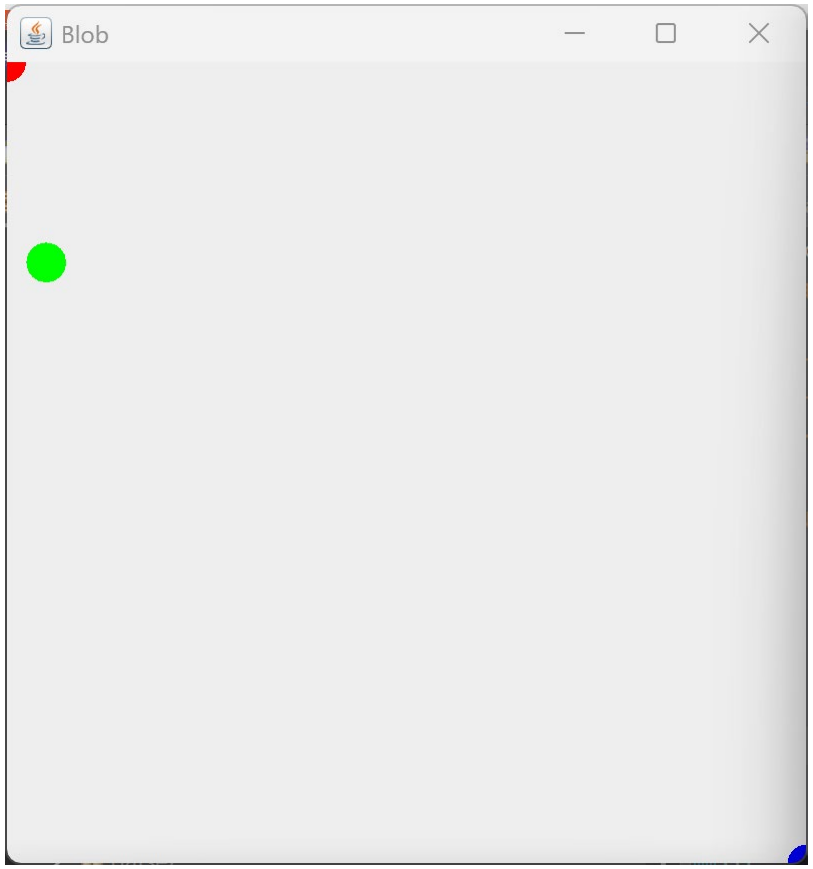
- Aufgabenüberblick
  - Erzeugung von Testdaten für statistische Operationen und Machine Learning
  - Zwei Strategien zur Erzeugung der Testdaten „Blobs“ und „Moons“
  - Verwendung des Strategiemusters
  - Visualisierung der Datensätze mittels `drawing` Paket (Paket ist bereits vorhanden)
  - Clustering mittels Standard-Algorithmus k-Means-Clustering (Nutzung einer Standard-Bibliothek)





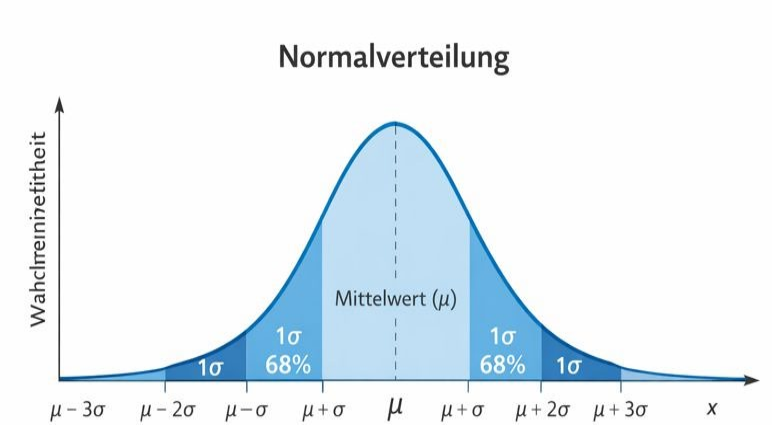
Vorlage

- Es gibt eine Vorlage `TemplateAssignment1.java`, die die Verwendung des `drawing`-Pakets zum Zeichnen von Punkten demonstriert
  - In dem Fenster werden drei Punkte eingezeichnet mit Durchmesser 20px (Achtung: später Durchmesser nur 6px)
  - Der erste und letzte Punkt sind nur teilweise sichtbar, da Randpunkte, deshalb später die Datenpunkte mit Abstand 20px zu allen Rändern skalieren



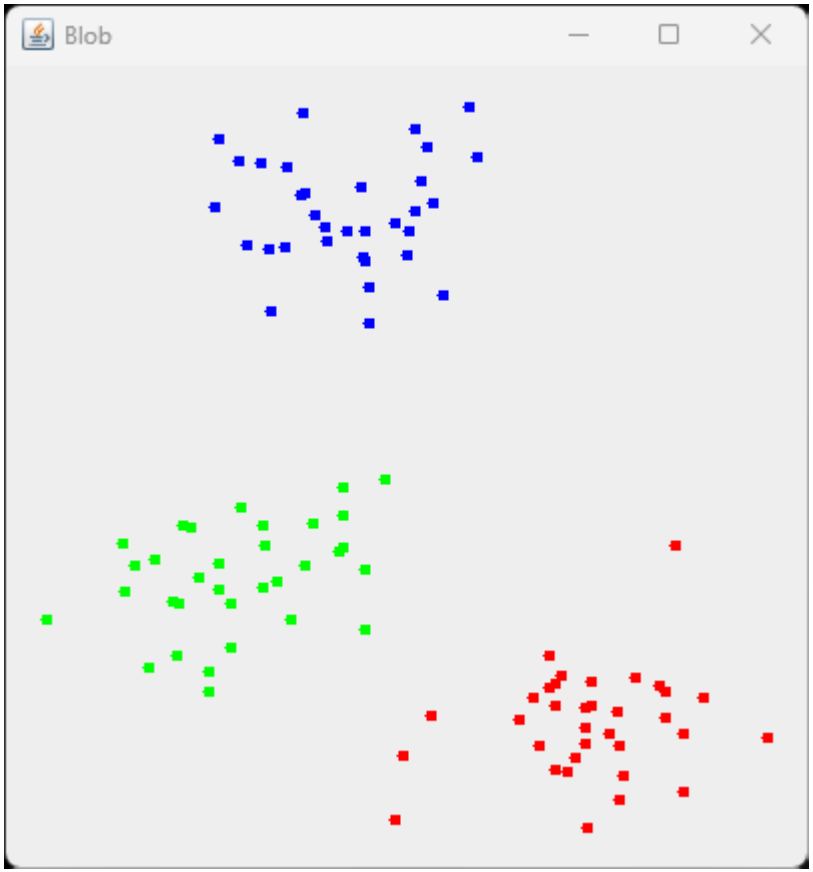
Normalverteilung (Gaußverteilung)

- Glockenkurve, die eine Häufigkeitsverteilung von Werten zeigt
- Im Alltag folgen viele Dinge einer Gaußverteilung, z. B.
  - Körpergröße
  - Notenverteilung
- Eigenschaften
  - Mittelwert (μ) - Durchschnitt bzw. Mitte der Kurve
  - Standardabweichung (σ) – bestimmt die Breite der Kurve (= Streuung)
  - Bei einer normierten Verteilung (= Standard-Normalverteilung) ist μ = 0 und σ = 1



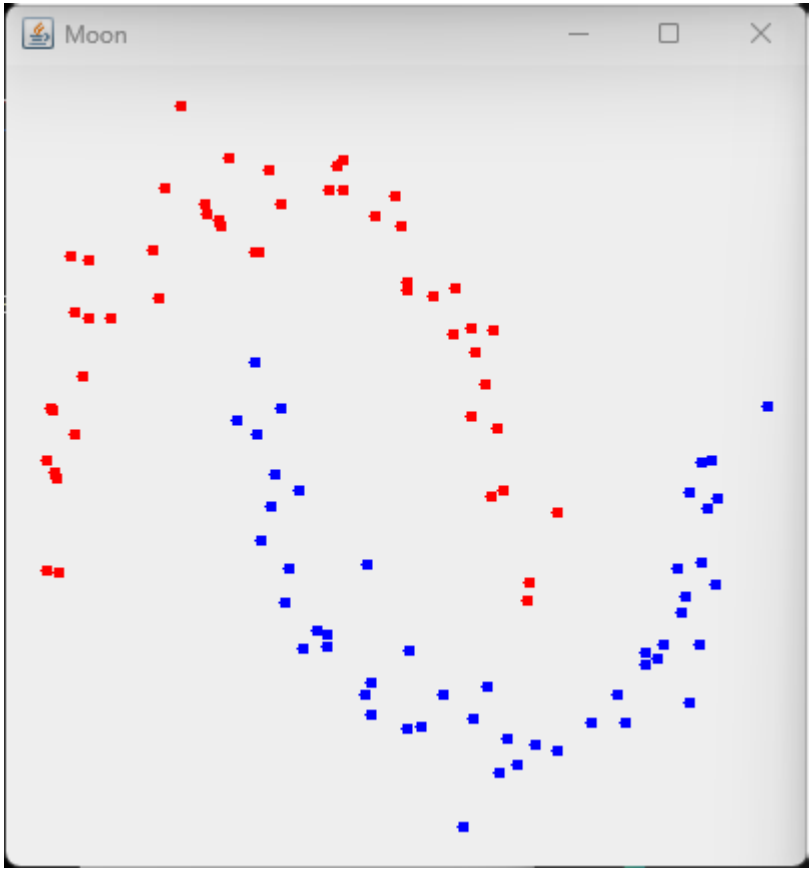
Blobs

- Cluster von Datenpunkten bildet sich um verschiedene Zentren
  - Datenpunkte sind in einer Normalverteilung um jedes Zentrum angeordnet
  - Für die Zentren werden zufällige Punkte innerhalb vorgegebener Grenzen erzeugt



Moons

- Cluster von Datenpunkten bildet sich um eine sinus/cosinus-Kurve
  - Die Datenpunkte werden mittels Normalverteilung um die Kurve verstreut
  - Die Kurven sind ineinandergreifende Halbkreise (Half Moons)

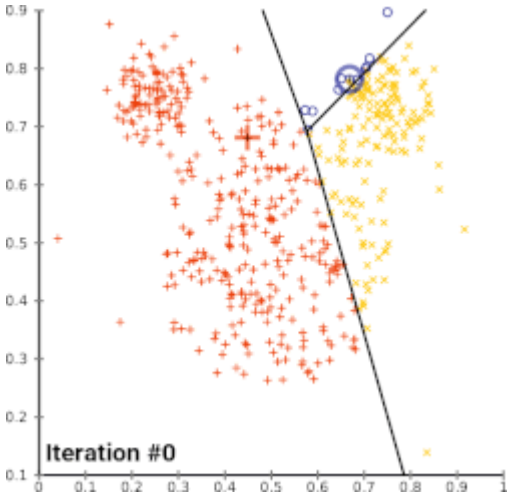


Vorgaben

- Datenpunkt (`DataPoint.java`) implementiert das Interface `Clusterable` aus `org.apache.commons.math3.ml.clustering.Clusterable;`
  - Enthält `x (double), y (double), label (int), clusterNo (int)`
  - `x` und `y` stellen die Koordinaten des Punktes dar
  - `label` ist die Zuordnung zu einem Cluster (0 bis Anzahl Cluster-1) während der **Erzeugung** der Daten
  - `clusterNo` ist die Zuordnung zu einem Cluster durch einen **Clustering-Algorithmus**

Vorgaben

https://de.wikipedia.org/wiki/K-Means-Algorithmus



- Datensatz (`Dataset.java`)
  - Verwaltet eine Liste von Datenpunkten
  - Nutzt eine Strategie zur Erzeugung von Datenpunkten
  - Erzeugt standardmäßig 100 Datenpunkte
  - Es können aber auch mehr Datenpunkte erzeugt werden
  - Enthält eine Methode `cluster(…)` die ein Clustering mittels `KMeansPlusPlus` (Apache Commons Math) mit den Parametern `k=3` bzw. `k=2` und 1.000 Iterationen analog User Guide durchführt; beim Durchlaufen der geclusterten Punkte werden die ermittelten Ergebnisse (= Cluster-Nummern) im Datapoint (`clusterNo`) gespeichert https://commons.apache.org/proper/commons-math/userguide/ml.html
  - Die Bibliothek commons-math3-3.6.1.jar ist bereits im Vorlage-Projekt eingebunden
    Doku https://commons.apache.org/proper/commons-math/javadocs/api-3.6.1/index.html
  - Anstelle eines LocationWrapper soll die eigene Klasse `DataPoint` verwendet werden

Vorgaben

- Blobs:
  - Die **Anzahl der Cluster**, die **Standardabweichung** und der **Seed** für den Zufallsgenerator sollen konfigurierbar sein
  - Die Breite und Höhe des Bereichs, in dem die Zentren liegen sollen, ist standardmäßig **15,** d.h. die y und y-Koordinaten der Zentren liegen im Intervall [0; 15]
  - Beispielparameter: Anzahl Cluster = 3, σ = 0,5 und Seed = 0 (siehe Diagramm auf Folie 5), d.h. mit einer Normalverteilung μ = 0 und σ = 0,5 werden die x- und y-Daten um die Zentren herum verteilt
- Moons
  - Die **Standardabweichung** und der **Seed** für den Zufallsgenerator sollen konfigurierbar sein
  - Es werden immer zwei Halb-Monde erstellt
  - Beispielparameter: σ = 0,1 und Seed = 0 (siehe Diagramm auf Folie 6), d.h. im Intervall $[0, \pi]$ werden `size/2` (= Anzahl) Punkte für den ersten Halbmond erzeugt.
    Die x- und y-Koordinaten des ersten Halbkreises werden mittels Formel $(x, y) = (\cos t, \sin t),\ t \in [0, \pi]$ berechnet. Mit einer Normalverteilung μ = 0 und σ = 0,1 werden die x- und y-Daten etwas „verrauscht“.
  - Zweiter Halbmond: $(x, y) = (1 - \cos t, 1 - \sin t - 0{,}5),\ t \in [0, \pi]$ Verrauschung wie oben

Vorgaben

- Scatter-Diagramm (`ScatterDiagram.java`)
  - Stellt einen Datensatz mittels `drawing`-Paket mittels einer `show()`-Methode dar
  - Nutzt unterschiedliche Farben zur Darstellung der Cluster (maximal 3 Farben rot, blau und grün, wenn die Farben ausgeschöpft sind, wieder mit Farben von vorne beginnen)
  - Skaliert das Diagramm, so dass es die komplette Fläche des Fensters nutzt, wobei ein Innenabstand von 20px auf jeder Seite beibehalten wird
  - Die Y-Achse soll invertiert werden (kleine Y-Werte werden unten, große oben angezeigt)
  - Beim Erzeugen des Diagramms soll der Titel angegeben werden; es soll auch entschieden werden, ob die Farben anhand des `label` oder der `clusterNo` des Datenpunktes angezeigt werden

Interface (Strategie)

- Die Strategien sollen folgendes Interface implementieren, hierbei ist `size` die Anzahl der zu erzeugende Punkte

```
public interface DataGenerator {
    List<DataPoint> generate(int size);
}
```

  Eine Beschreibung des Strategie-Musters ist im Import-Java-Projekt enthalten: `Strategie-Muster.pdf`

- Ein Zufallswert für eine normierte Gaußverteilung ($\mu = 0$, $\sigma = 1$) kann mittels folgendem Quellcodefragment erzeugt werden

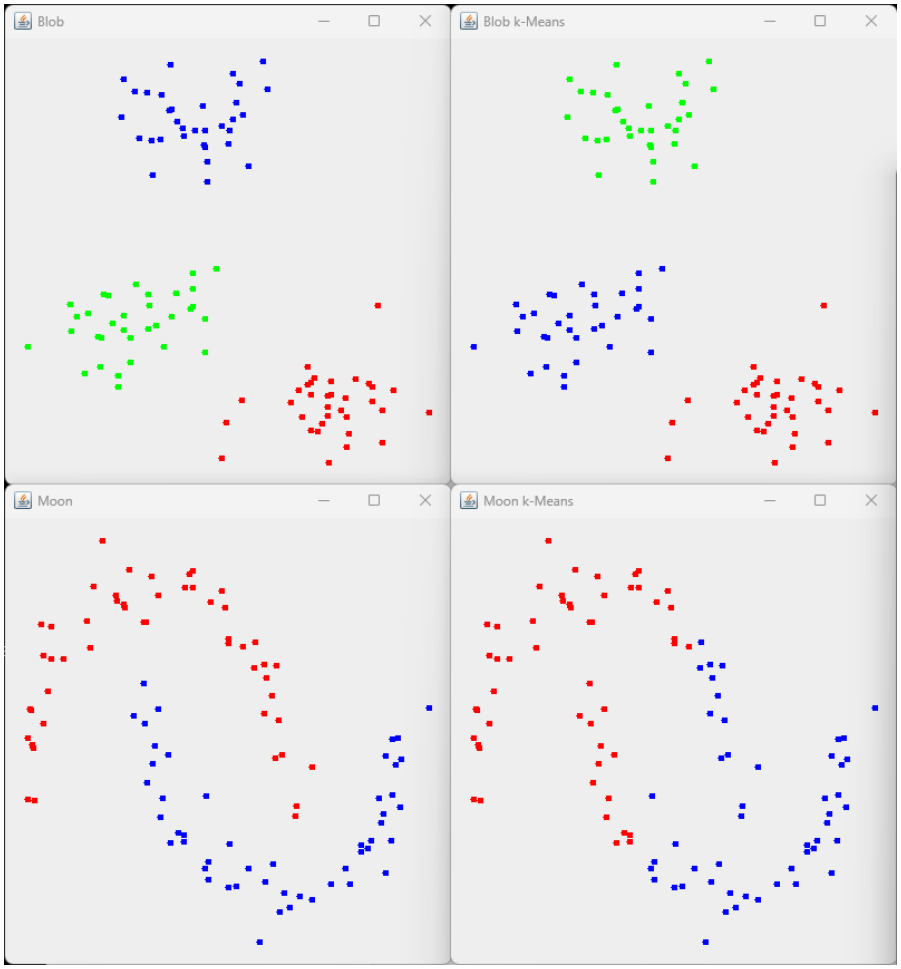
```
import java.util.Random;

Random randomNumberGenerator = new Random(0);
double number = randomNumbergenerator.nextGaussian()
// Hinweis: Anderes σ mittels Multiplikation
// double number = σ * randomNumbergenerator.nextGaussian()
```

Main und Testfälle

- Eine `Main`-Klasse mit `main`-Methode erzeugt und zeigt die nebenstehenden 4 Fenster mit den generierten bzw. geclusterten Daten an

  k-Means oben rechts mit k = 3
  k-Means unten rechts mit k = 2



- Erstellung von drei Testfällen:

1. Wurden 100 Datenpunkte für Blobs erzeugt?
2. Wurden 100 Datenpunkte für Moon erzeugt?
3. Prüfung, ob Moon-Daten in genau zwei `label`- Kategorien eingeordnet sind

Vorgehen

- Vorgehen
  - Importieren des Java-Projekts und Umbenennen mit eigenem Namen
  - Anlegen einer Klasse für einen einzelnen Datenpunkt
  - Verwenden der Vorlage, um ein Scatter Diagramm mithilfe der Datenpunkte darzustellen. Das Scatter Diagramm soll unterschiedliche Farben je nach Label des Datenpunktes verwenden
  - Anlegen eines Datensatzes der eine Liste von Datenpunkten verwaltet und eine Strategie zur Generierung der Daten verwendet
  - Implementieren der Strategien
  - Verwendung des Datensatzes im Scatter Diagramm
  - Skalieren des Scatter Diagramms
  - Schreiben der `main`-Methode zur Anzeige aller vier Fenster, Schreiben der Testfälle unter Verwendung von `public`-Konstanten aus `Main`.